Lab 4 Report:

Surpass Human Performance in Fashion MNIST Classificaion

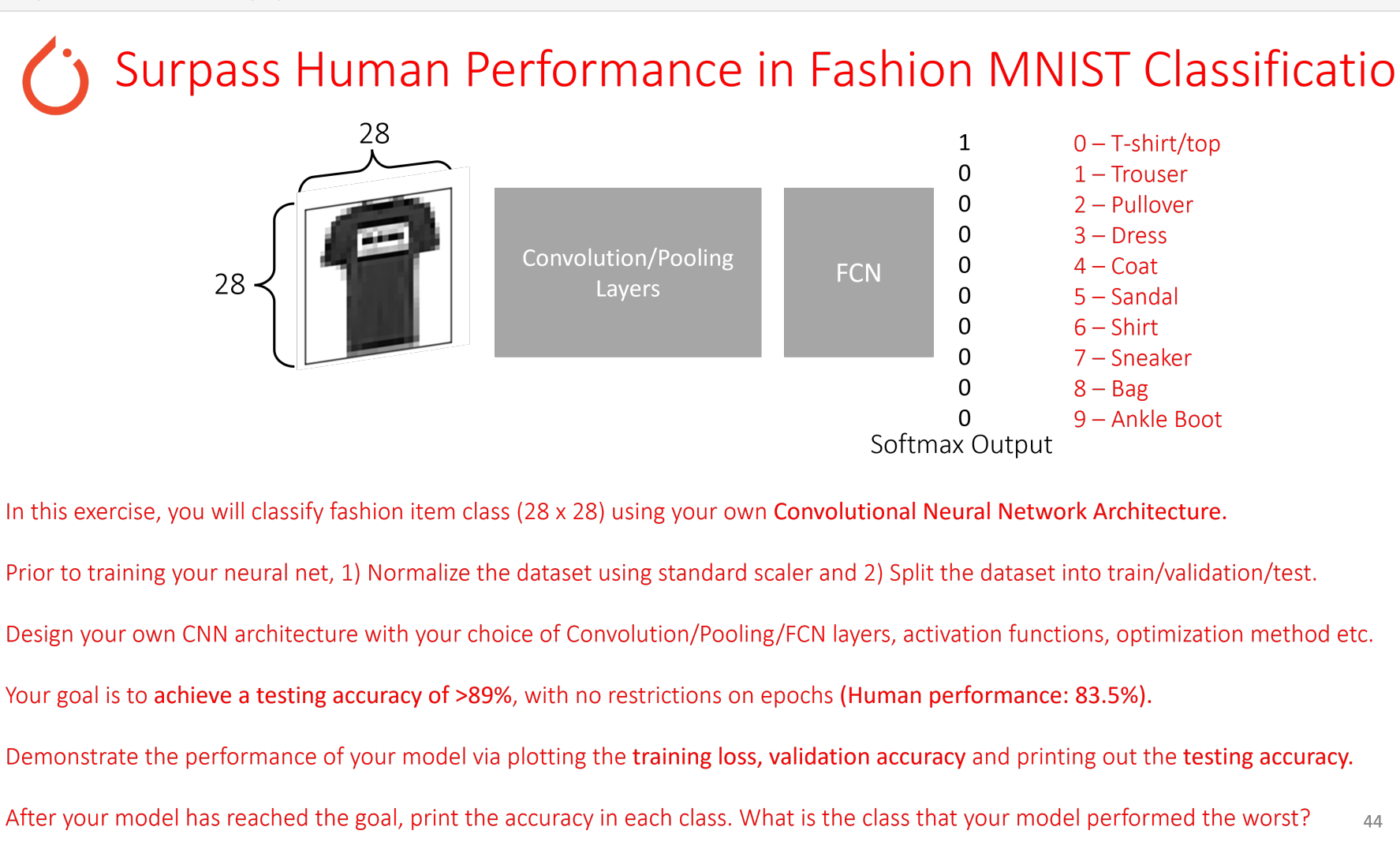
Name: Josh Lando w/ Claude

```
In [23]: %matplotlib inline
         import matplotlib.pyplot as plt
         import torch
         import numpy as np

         np.random.seed(42)
         torch.manual_seed(42)

Out[23]: <torch._C.Generator at 0x10f2ed910>
```

```
In [24]: from IPython.display import Image # For displaying images in colab jupyter cell
```

```
In [25]: Image('lab4_exercise.png', width = 1000)
```

Out[25]:



Surpass Human Performance in Fashion MNIST Classification

In this exercise, you will classify fashion item class (28 x 28) using your own **Convolutional Neural Network Architecture.**

Prior to training your neural net, 1) Normalize the dataset using standard scaler and 2) Split the dataset into train/validation/test.

Design your own CNN architecture with your choice of Convolution/Pooling/FCN layers, activation functions, optimization method etc.

Your goal is to **achieve a testing accuracy of >89%**, with no restrictions on epochs **(Human performance: 83.5%).**

Demonstrate the performance of your model via plotting the **training loss, validation accuracy** and printing out the **testing accuracy.**

After your model has reached the goal, print the accuracy in each class. What is the class that your model performed the worst?

Prepare Data

```
In [26]: device = torch.device("mps" if torch.mps.is_available() else "cpu")
         print(f"Using device: {device}")

Using device: mps
```

```
In [27]: # Load Fashion-MNIST Dataset in Numpy

         # 10000 training features/targets where each feature is a greyscale image with shape (28, 28)
         train_features = np.load('fashion_mnist_train_features.npy')
         train_targets = np.load('fashion_mnist_train_targets.npy')

         # 1000 testing features/targets
         test_features = np.load('fashion_mnist_test_features.npy')
         test_targets = np.load('fashion_mnist_test_targets.npy')

         # Let's see the shapes of training/testing datasets
         print("Training Features Shape: ", train_features.shape)
         print("Training Targets Shape: ", train_targets.shape)
         print("Testing Features Shape: ", test_features.shape)
         print("Testing Targets Shape: ", test_targets.shape)

Training Features Shape:  (10000, 28, 28)
Training Targets Shape:  (10000,)
Testing Features Shape:  (1000, 28, 28)
Testing Targets Shape:  (1000,)
```

```
In [28]: # Visualizing up to 10 training features (samples)
         label_names = [
             'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
             'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'
         ]
         n_classes = 10

         def plot(features, targets, max_plots=4):
             n = min(len(features), max_plots)
             cols = min(5, n)
             rows = (n + cols - 1) // cols

             plt.figure(figsize=(cols * 3, rows * 3))
             for i in range(n):
                 plt.subplot(rows, cols, i + 1)
                 plt.imshow(features[i], cmap='Greys')
                 plt.axis('off')
                 plt.title(f'{label_names[targets[i]]}')
             plt.tight_layout()

         plot(train_features, train_targets)
```

Ankle boot
T-shirt/top
T-shirt/top
Dress

```
In [29]: # Print class distribution percentages

         for dataset_name, targets in [("Training", train_targets), ("Test", test_targets)]:
             print(f"\n{dataset_name} Targets Distribution:")
             unique, counts = np.unique(targets, return_counts=True)
             for label, count in zip(unique, counts):
                 percentage = (count / len(targets)) * 100
                 print(f"  Class {label}: {count} samples ({percentage:.2f}%)")

Training Targets Distribution:
  Class 0: 942 samples (9.42%)
  Class 1: 1027 samples (10.27%)
  Class 2: 1016 samples (10.16%)
  Class 3: 1019 samples (10.19%)
  Class 4: 974 samples (9.74%)
  Class 5: 989 samples (9.89%)
  Class 6: 1021 samples (10.21%)
  Class 7: 1022 samples (10.22%)
  Class 8: 990 samples (9.90%)
  Class 9: 1000 samples (10.00%)

Test Targets Distribution:
  Class 0: 107 samples (10.70%)
  Class 1: 105 samples (10.50%)
  Class 2: 111 samples (11.10%)
  Class 3: 93 samples (9.30%)
  Class 4: 115 samples (11.50%)
  Class 5: 87 samples (8.70%)
  Class 6: 97 samples (9.70%)
  Class 7: 95 samples (9.50%)
  Class 8: 95 samples (9.50%)
  Class 9: 95 samples (9.50%)
```

```
In [30]: # Import StandardScaler from scikit-learn

         from sklearn.preprocessing import StandardScaler

         # First flatten each image into 784 to convert features from 3D -> 2D arrays
         train_features_flat = train_features.reshape((10000, 28 * 28))
         test_features_flat = test_features.reshape((1000, 28 * 28))

         # Use standard scaler to scale the flattened images
         # Reshape back to 28 x 28 since CNNs accept 2D tensors as inputs
         scaler = StandardScaler()
         train_features = scaler.fit_transform(train_features_flat).reshape((10000, 28, 28))
         test_features = scaler.fit_transform(test_features_flat).reshape((1000, 28, 28))

         # zero out train features
         # train_features = np.zeros_like(train_features)
```

```
In [31]: plot(train_features, train_targets)
```

Ankle boot
T-shirt/top
T-shirt/top
Dress

```
In [32]: # Taking the first 100 training features/targets as validation dataset
         validation_features = train_features[:1000]
         validation_targets = train_targets[:1000]

         # Taking the remaining 900 training features/targets as training dataset
         train_features = train_features[1000:]
         train_targets = train_targets[1000:]
```

```
In [33]: # Reshape train/validation/test features to according to (N, channels, height, width) where N is number of samples

         train_features = np.reshape(train_features, (9000, 1, 28, 28))
         validation_features = np.reshape(validation_features, (1000, 1, 28, 28))
         test_features = np.reshape(test_features, (1000, 1, 28, 28))
```

Define Model

```
In [34]: # Define your CNN architecture here

         class CNNModel(torch.nn.Module):

             def __init__(self):

                 super(CNNModel, self).__init__()

                 # First convolution layer (1 channel -> 16 channels, preserve original dimension by adding padding = 2)
                 self.cnn1 = torch.nn.Conv2d(in_channels=1, out_channels=18,
                                       kernel_size=3, stride=1, padding=0)

                 # First max pooling layer with kernel size = 2
                 self.maxpool1 = torch.nn.MaxPool2d(kernel_size=2)

                 # # Second convolution layer (16 channel -> 32 channels, preserve dimension by adding padding = 2)
                 # self.cnn2 = torch.nn.Conv2d(in_channels=16, out_channels=32,
                 #                        kernel_size=3, stride=1, padding=0)

                 # # Second max pooling layer with kernel size = 2
                 # self.maxpool2 = torch.nn.MaxPool2d(kernel_size=2)

                 self.dropout = torch.nn.Dropout(p=.1)

                 # Fully connected layer that takes the flattened output of maxpool2 (32, 7 ,7) -> (1568) and output 10 classes
                 self.fc1 = torch.nn.Linear(3042, 200)
                 self.ac1 = torch.nn.ReLU()
                 self.fc2 = torch.nn.Linear(200, 10)

             def forward(self, x):

                  # input image -> conv1 -> relu -> maxpool1
                 conv1_out = torch.nn.functional.relu(self.cnn1(x))
                 pool1_out = self.maxpool1(conv1_out)

                 # # maxpool1 output -> conv2 -> relu -> maxpool2
                 # conv2_out = torch.nn.functional.relu(self.cnn2(pool1_out))
                 # pool2_out = self.maxpool2(conv2_out)

                 # flatten the maxpool2 output to be used as input into FCN layer
                 fcn_input = pool1_out.view(pool1_out.size(0), -1)
                 fcn_input = self.dropout(fcn_input)

                 # Use the raw output of the fully connected layer as the final output
                 fc1_out = self.fc1(fcn_input)
                 fc1_out = self.ac1(fc1_out)
                 fc1_out = self.dropout(fc1_out)

                 output = self.fc2(fc1_out)

                 return output
```

Select Hyperparameters

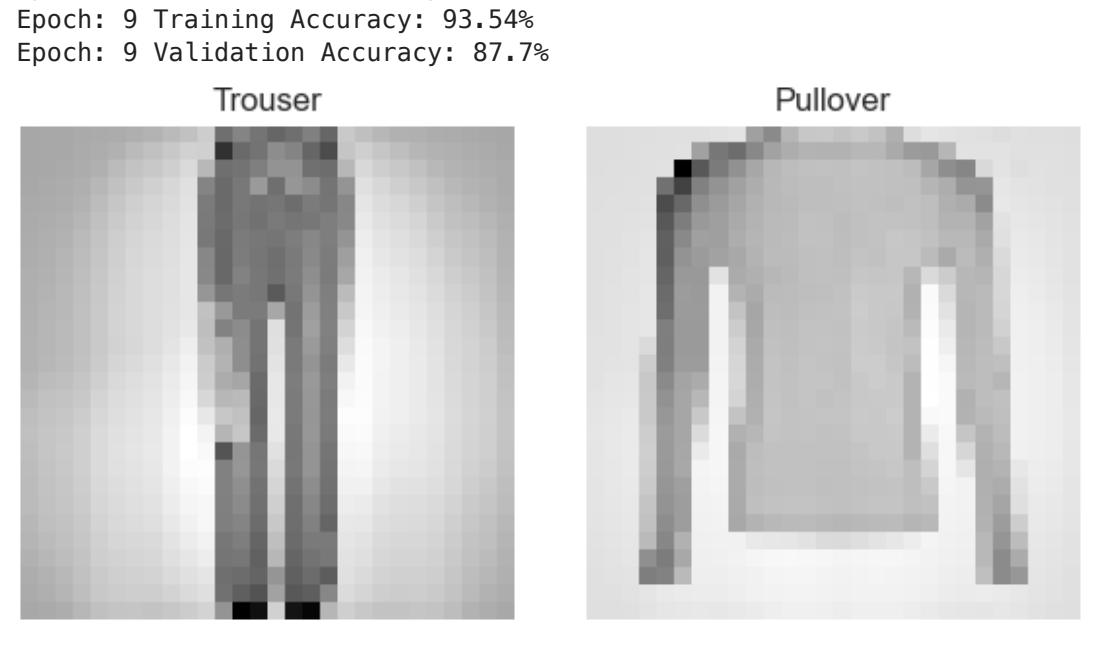
```
In [35]: torch.manual_seed(42)
         from torch.optim.lr_scheduler import StepLR

         # Define the model class
         model = CNNModel().to(device)

         # Define learning rate, epochs and batchsize (for mini-batch gradient)
         learning_rate_batch = 0.003
         learning_rate_full = 0.003
         weight_decay = 5e-4
         epochs = 10
         batchsize = 100
         switch_full = 1

         # CrossEntropyLoss as loss function and Adam as optimizer
         # NOTE: CrossEntropyLoss automatically applies softmax to the output layer
         # so no need to apply manual softmax to the output layer in forward function
         loss_func = torch.nn.CrossEntropyLoss()
         optimizer_batch = torch.optim.Adam(model.parameters(), lr = learning_rate_batch, weight_decay=weight_decay)
         optimizer_full = torch.optim.Adam(model.parameters(), lr = learning_rate_full, weight_decay=weight_decay)
         scheduler = StepLR(optimizer_full, step_size=10, gamma=0.1)

         # Print modele specific. Add .cuda() to move your model to GPU
         model

Out[35]: CNNModel(
           (cnn1): Conv2d(1, 18, kernel_size=(3, 3), stride=(1, 1))
           (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
           (dropout): Dropout(p=0.1, inplace=False)
           (fc1): Linear(in_features=3042, out_features=200, bias=True)
           (ac1): ReLU()
           (fc2): Linear(in_features=200, out_features=10, bias=True)
         )
```

Identify Tracked Values

```
In [36]: # Placeholders for training loss and validation accuracy during training
         # Using empty Python list as train_loss_list since we will be appending loss value per iteration
         # each iteration = single fwd/bwd pass of a training mini-batch

         train_loss_list = []
         train_accuracy_list = np.zeros((epochs,))
         validation_accuracy_list = np.zeros((epochs,))
```

Train Model

```
In [37]: import tqdm # Use "for epoch in tqdm.trange(epochs):" to see the progress bar

         # Convert train/validation/test sets into torch tensors
         # Features -> floating numbers, Target labels -> int64 format
         train_inputs = torch.from_numpy(train_features).float().to(device)
         train_targets = torch.from_numpy(train_targets).long().to(device)

         validation_inputs = torch.from_numpy(validation_features).float().to(device)
         validation_targets = torch.from_numpy(validation_targets).long().to(device)

         testing_inputs = torch.from_numpy(test_features).float().to(device)
         testing_targets = torch.from_numpy(test_targets).long().to(device)

         # Use torch.split() function to split the training inputs/targets into mini-batches
         # See documentation of torch.split() https://pytorch.org/docs/stable/generated/torch.split.html
         train_batches_features = torch.split(train_inputs, batchsize)
         train_batches_targets = torch.split(train_targets, batchsize)

         print("Number of batches:", len(train_batches_features))
         print("Shape of first batch:", train_batches_features[0].shape)

         # length of train_batches_features = total number of mini-batches in the training set
         batch_split_num = len(train_batches_features)

         # Training Loop ---------------------------------------------------------------------------------------
         switch = True
         batch = True

         for epoch in range(epochs):

             # For each mini-batch number k, grab k-th training feature mini-batch and target mini-batch
             # and perform fwd/bwd pass on the network


             if batch:
                 for k in range(batch_split_num):

                     optimizer_batch.zero_grad()

                     if epoch == 0 and k == 0:
                         plot(train_batches_features[k][:2, 0, :, :].cpu().numpy(), train_batches_targets[k][:2].cpu().numpy())

                     train_batch_outputs = model(train_batches_features[k])

                     loss = loss_func(train_batch_outputs, train_batches_targets[k])

                     train_loss_list.append(loss.item())

                     loss.backward()

                     optimizer_batch.step()
             else:
                 optimizer_full.zero_grad()

                 train_outputs = model(train_inputs)

                 loss = loss_func(train_outputs, train_targets)

                 train_loss_list.append(loss.item())

                 loss.backward()

                 optimizer_full.step()

                 scheduler.step()

             # Compute Train & Validation Accuracy ---------------------------------------------------------------------

             with torch.no_grad():

                 train_outputs = model(train_inputs)

                 correct = (torch.argmax(train_outputs, dim=1) ==
                           train_targets).type(torch.FloatTensor)

                 print("Epoch: "+ str(epoch),
                       "Training Accuracy: " + str(np.round(correct.mean().numpy() * 100, 2)) + '%', flush=True)

                 train_accuracy_list[epoch] = correct.mean()

                 if batch and correct.mean() > switch_full:
                     batch = False
                     print("switching to full batch training")

                 validation_outputs = model(validation_inputs)

                 correct = (torch.argmax(validation_outputs, dim=1) ==
                           validation_targets).type(torch.FloatTensor)

                 print("Epoch: "+ str(epoch),
                       "Validation Accuracy: " + str(np.round(correct.mean().numpy() * 100, 2)) + '%', flush=True)

                 validation_accuracy_list[epoch] = correct.mean()

Number of batches: 90
Shape of first batch: torch.Size([100, 1, 28, 28])
Epoch: 0 Training Accuracy: 85.86%
Epoch: 0 Validation Accuracy: 85.5%
Epoch: 1 Training Accuracy: 89.17%
Epoch: 1 Validation Accuracy: 86.1%
Epoch: 2 Training Accuracy: 89.1%
Epoch: 2 Validation Accuracy: 85.7%
Epoch: 3 Training Accuracy: 91.1%
Epoch: 3 Validation Accuracy: 85.8%
Epoch: 4 Training Accuracy: 91.61%
Epoch: 4 Validation Accuracy: 86.8%
Epoch: 5 Training Accuracy: 93.61%
Epoch: 5 Validation Accuracy: 87.7%
Epoch: 6 Training Accuracy: 91.97%
Epoch: 6 Validation Accuracy: 87.1%
Epoch: 7 Training Accuracy: 92.9%
Epoch: 7 Validation Accuracy: 87.4%
Epoch: 8 Training Accuracy: 93.57%
Epoch: 8 Validation Accuracy: 86.4%
Epoch: 9 Training Accuracy: 93.54%
Epoch: 9 Validation Accuracy: 87.7%
```

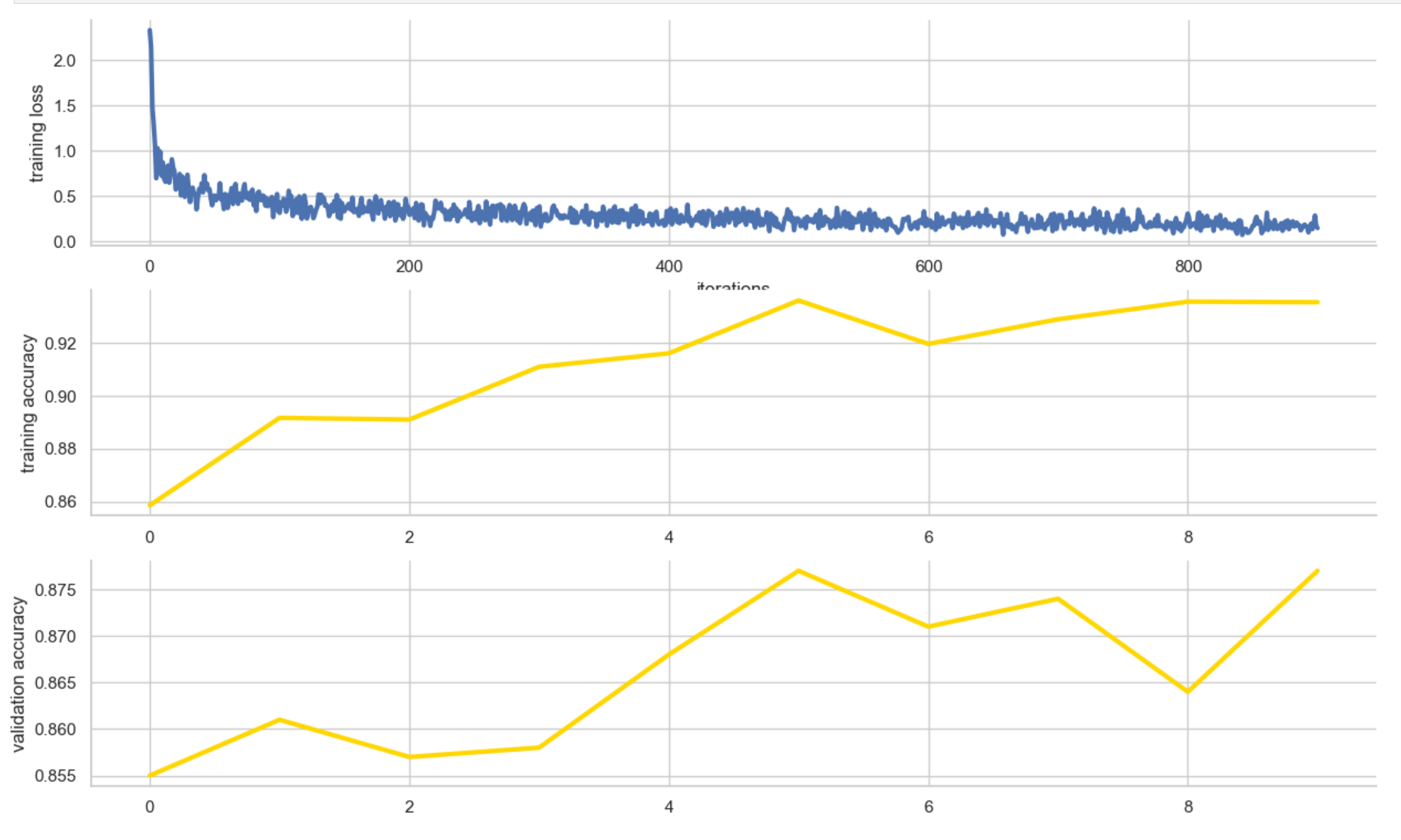


Visualize & Evaluate Model

```
In [38]: print("First training loss value: ", train_loss_list[0])
         print("what it should be: ", -np.log(1/10))
         print("Last training loss value: ", train_loss_list[-1])

First training loss value:  2.333061695098877
what it should be:  2.3025850929940455
Last training loss value:  0.14866699278354645
```

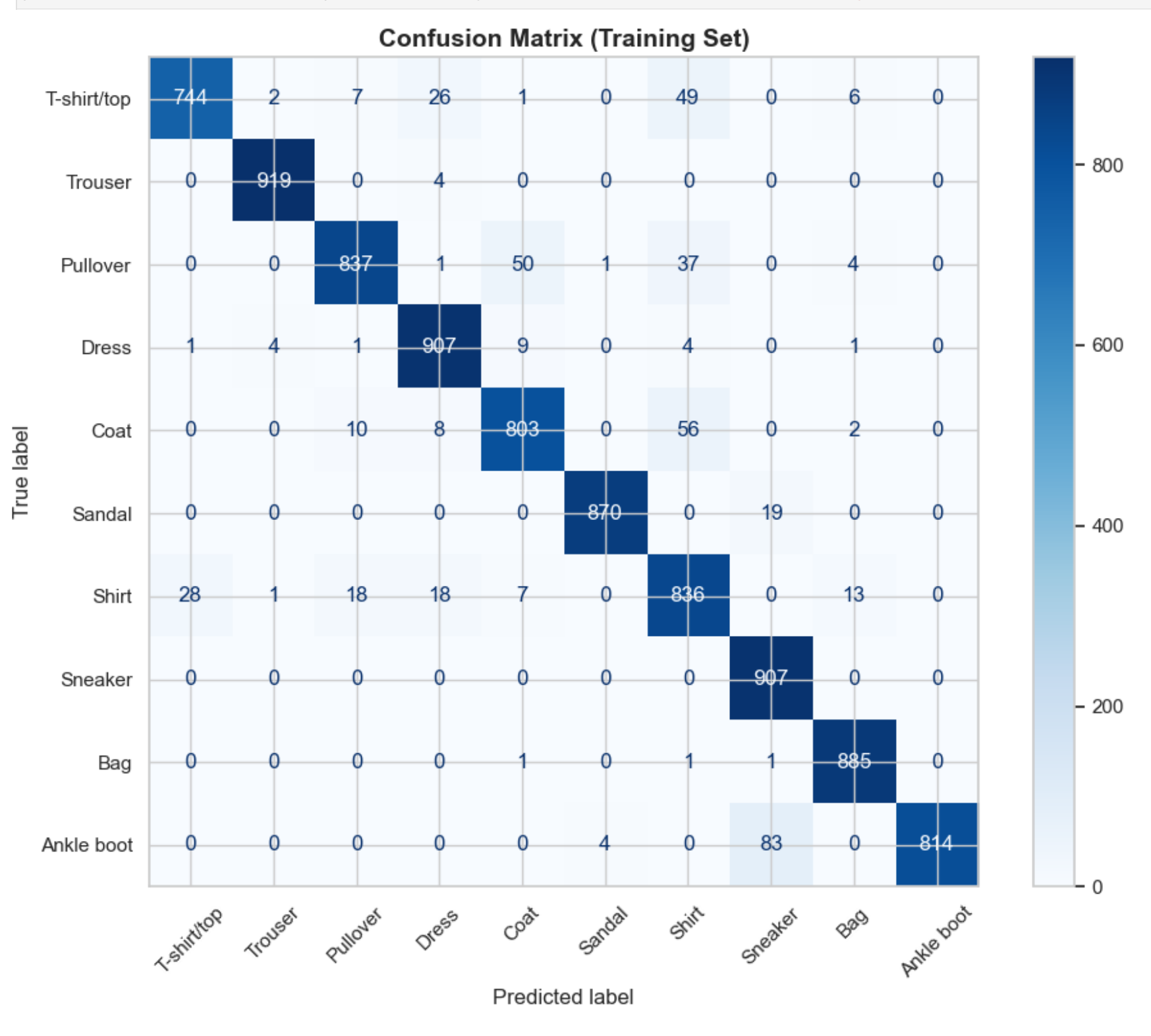
```
In [39]: # Seaborn for prettier plot

         import seaborn as sns

         sns.set(style = 'whitegrid', font_scale = 1)
```

```
In [40]: # Visualize training loss

         plt.figure(figsize = (15, 9))

         plt.subplot(3, 1, 1)
         plt.plot(train_loss_list, linewidth = 3)
         plt.ylabel("training loss")
         plt.xlabel("iterations")
         sns.despine()

         plt.subplot(3, 1, 2)
         plt.plot(train_accuracy_list, linewidth = 3, color = 'gold')
         plt.ylabel("training accuracy")
         sns.despine()

         plt.subplot(3, 1, 3)
         plt.plot(validation_accuracy_list, linewidth = 3, color = 'gold')
         plt.ylabel("validation accuracy")
         sns.despine()
```



```
In [41]: # Compute the testing accuracy

         # YOUR CODE HERE
         with torch.no_grad():

             y_pred_test = model(testing_inputs)

             correct = (torch.argmax(y_pred_test, dim=1) == testing_targets).type(torch.FloatTensor)

             print("Testing Accuracy: " + str(correct.mean().numpy()))

Testing Accuracy: 0.868
```

```
In [42]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

         def plot_confusion_matrix(inputs, targets, title='Confusion Matrix'):
             model.eval()
             with torch.no_grad():
                 outputs = model(inputs)
                 preds = torch.argmax(outputs.cpu(), dim=1).numpy()
             model.train()

             true = targets.cpu().numpy()
             cm = confusion_matrix(true, preds)

             fig, ax = plt.subplots(figsize=(10, 8))
             disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_names)
             disp.plot(ax=ax, colorbar=True, xticks_rotation=45, cmap='Blues')
             ax.set_title(title, fontsize=14, fontweight='bold')
             plt.tight_layout()
             plt.show()

         plot_confusion_matrix(train_inputs, train_targets, title='Confusion Matrix (Training Set)')
```



Confusion Matrix (Training Set)

True label	T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot
T-shirt/top	744	2	7	26	1	0	49	0	6	0
Trouser	0	919	0	4	0	0	0	0	0	0
Pullover	0	0	837	1	50	1	37	0	4	0
Dress	1	4	1	907	9	0	4	0	1	0
Coat	0	0	10	8	803	0	56	0	2	0
Sandal	0	0	0	0	0	870	0	19	0	0
Shirt	28	1	18	18	7	0	836	0	13	0
Sneaker	0	0	0	0	0	0	0	907	0	0
Bag	0	0	0	0	1	0	1	1	885	0
Ankle boot	0	0	0	0	0	4	0	83	0	814

```
In [43]: def plot_incorrect(inputs, targets, title='Incorrect Predictions', max_plots=20):
             model.eval()
             with torch.no_grad():
                 outputs = model(inputs)
                 preds = torch.argmax(outputs, dim=1).cpu().numpy()
             model.train()

             true = targets.cpu().numpy()
             imgs = inputs.cpu().numpy()

             wrong_idx = np.where(preds != true)[0]
             print(f"Total incorrect predictions: {len(wrong_idx)}")

             n = min(max_plots, len(wrong_idx))
             cols = 5
             rows = (n + cols - 1) // cols

             plt.figure(figsize=(cols * 3, rows * 3.5))
             for i, idx in enumerate(wrong_idx[:n]):
                 plt.subplot(rows, cols, i + 1)
                 plt.imshow(imgs[idx, 0], cmap='Greys')
                 plt.axis('off')
                 plt.title(f'True: {label_names[true[idx]]}\nPred: {label_names[preds[idx]]}', fontsize=8)
             plt.suptitle(f'{title} ({len(wrong_idx)} total)', fontsize=13, fontweight='bold')
             plt.tight_layout()
             plt.show()

         plot_incorrect(testing_inputs, testing_targets, title='Incorrect Predictions (Test Set)')

Total incorrect predictions: 130
```

Incorrect Predictions (Test Set) (130 total)
True: Coat
Pred: Shirt
True: Sandal
Pred: Sneaker
True: Ankle boot
Pred: Sneaker
True: Coat
Pred: Pullover
True: Dress
Pred: Shirt
True: Pullover
Pred: Shirt
True: Coat
Pred: Shirt
True: Coat
Pred: Shirt
True: Pullover
Pred: Trouser
True: Dress
Pred: Coat
True: Ankle boot
Pred: Sneaker
True: Pullover
Pred: Coat
True: Dress
Pred: Trouser
True: Coat
Pred: Pullover
True: Ankle boot
Pred: Sneaker
True: T-shirt/top
Pred: Dress
True: T-shirt/top
Pred: Shirt
True: Shirt
Pred: Dress
True: Coat
Pred: Dress
True: Coat
Pred: Shirt